

Análisis de Dominio

API Restaurante La Parrilla Karen Becerra

En este análisis vemos qué datos necesita exponer la API del restaurante La Parrilla y de dónde vienen. La idea es que el frontend (la web o app del restaurante) necesita comunicarse con el backend para hacer cosas como crear una reserva, consultar el menú o ver el estado de un pedido. La API hace de puente entre los dos, y este análisis describe cómo está organizado ese puente.

Tablas de la base de datos que usa la API

La base de datos del restaurante tiene tres tablas principales que participan en las operaciones de la API:

Reservas — reservas

Es la tabla central de este análisis. Guarda cada reserva que hace un cliente: para qué mesa, cuándo, cuántas personas y en qué estado está (pendiente, confirmada, cancelada).

Mesas — mesas

Almacena la información de cada mesa del restaurante: su número, cuántas personas caben, dónde está (interior, terraza, zona privada) y si está disponible o no. La API la consulta antes de crear una reserva para validar que la mesa existe y tiene hueco.

Platos — platos

Contiene el menú: nombre, descripción, precio, categoría, alérgenos y si está disponible ese día. La API la usa para mostrar el menú y para validar los pedidos.

Pedidos e ítems — pedidos + items_pedido

Aquí pedidos guarda cada pedido de una mesa (estado y total), e items_pedido guarda los platos concretos de cada pedido con su cantidad y subtotal. Son dos tablas separadas porque un pedido puede tener varios platos.

Campos de la entidad Reserva

Campo	Tipo en BBDD	¿Expuesto?	Motivo / Notas
id	INT (PK)	Solo en respuesta	Lo genera automáticamente la BBDD. El cliente nunca lo envía.
numero_mesa	INT (FK)	Sí	Es el recurso que se reserva. Va como parámetro en la URL: /mesas/{numeroMesa}/reservar.
nombre_cliente	VARCHAR(100)	Sí (obligatorio)	Identifica al titular de la reserva. El restaurante necesita saber a nombre de quién está la mesa.
telefono	VARCHAR(20)	Sí (opcional)	Para llamar si hay algún problema con la reserva. No es obligatorio.
email	VARCHAR(100)	Sí (opcional)	Para enviar confirmación por correo. Validamos que tenga formato de email.
num_comensales	INT	Sí (obligatorio)	Cuántas personas vienen a comer. Mínimo tiene que ser 1. No puede superar la capacidad de la mesa.
fecha_hora	DATETIME	Sí (obligatorio)	Cuándo es la reserva. Formato ISO 8601. Tiene que ser a fecha futura.
observaciones	TEXT	Sí (opcional)	Notas del cliente: alergias, etc...
estado	ENUM	Solo lectura para cliente	El backend lo pone a 'pendiente' al crear la reserva. Solo puede cambiarlo el staff.
created_at	TIMESTAMP	Solo en respuesta	Lo pone automáticamente la BBDD al insertar el registro. No se puede enviar ni modificar.
updated_at	TIMESTAMP	Solo en respuesta	Igual que created_at, se actualiza solo cuando cambia algún campo.

Validaciones y reglas de negocio

Son las reglas que el backend tiene que comprobar antes de guardar o modificar una reserva:

Campos obligatorios

- nombre_cliente no puede estar vacío.
- fecha_hora es obligatoria y tiene que ser una fecha futura (no se pueden reservar fechas del pasado).
- num_comensales es obligatorio y tiene que ser al menos 1.

Restricciones de formato y valor

- fecha_hora debe seguir el formato ISO 8601 (ejemplo: 2026-04-15T20:30:00Z).

- num_comensales tiene que ser un número entero positivo.
- email, si se envía, tiene que tener formato válido (que contenga @ y dominio).
- estado solo puede ser uno de estos tres valores: pendiente, confirmada o cancelada.

Lógica de negocio específica del restaurante

- Conflicto de horario: antes de crear la reserva, el backend mira si esa mesa ya tiene una reserva activa a la misma hora. Si la tiene, devuelve HTTP 409 (Conflicto) con un mensaje explicando el problema.
- Capacidad de la mesa: el backend comprueba que num_comensales no supere la capacidad de la mesa. Si se supera, devuelve HTTP 400 con el detalle del error.
- Estado inicial: toda reserva nueva se crea con estado = 'pendiente'. Solo el personal del restaurante puede cambiarlo a 'confirmada' o 'cancelada' usando un PATCH.
- Campos automáticos: id, created_at y updated_at nunca los manda el cliente. Los gestiona directamente la base de datos, y el backend los incluye en la respuesta para que el frontend los pueda usar.

4. Cómo fluyen los datos (ejemplo de reserva)

Para entender bien cómo encaja todo, aquí va el flujo completo de cuando un cliente crea una reserva:

- Primero el cliente rellena el formulario de reserva en la web (frontend) y pulsa 'reservar'
- Segundo el frontend ya hace una petición POST a /mesas/7/reservar enviando en el body: nombre, teléfono ,email ,número de comensales, fecha/hora y observaciones. No se envía el id ni el estado
- Después el backend recibe la petición y ya valida los campos
- Luego si hay algún error de validación responde HTTP 400 con JSON explicando qué campo está mal
- A continuación si la mesa ya está reservada a esa hora responde HTTP 409
- Más tarde si todo está bien se inserta la reserva en la base de datos con estado 'pendiente' y responde HTTP 201 con todos los datos de la reserva, incluyendo el id y el timestamp generado automáticamente